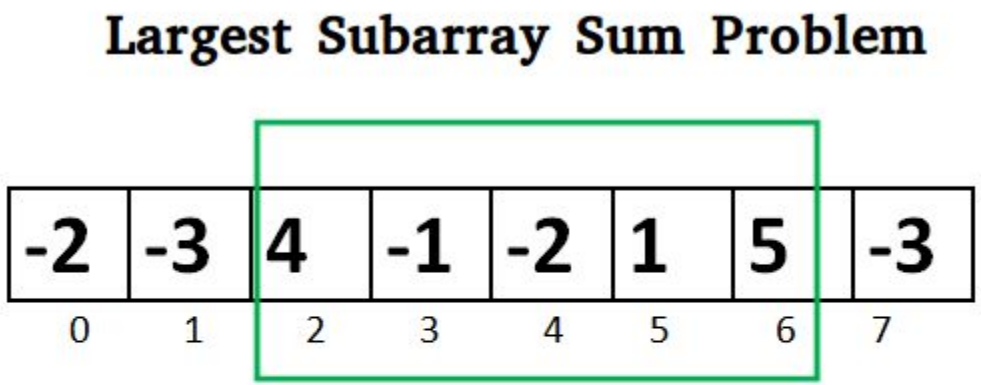
Divide and Conquer (Cont…)

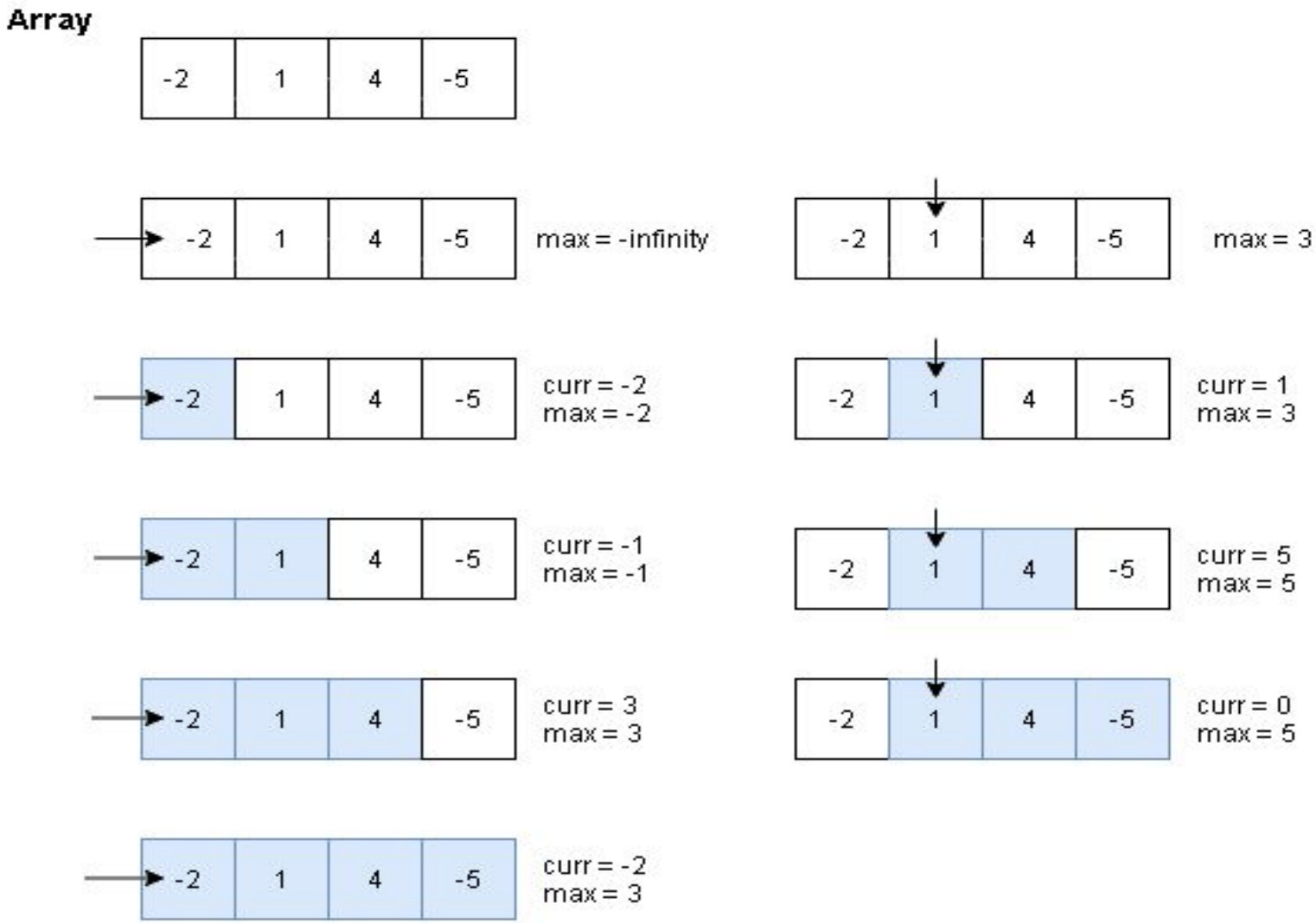
Maximum Subarray Sum



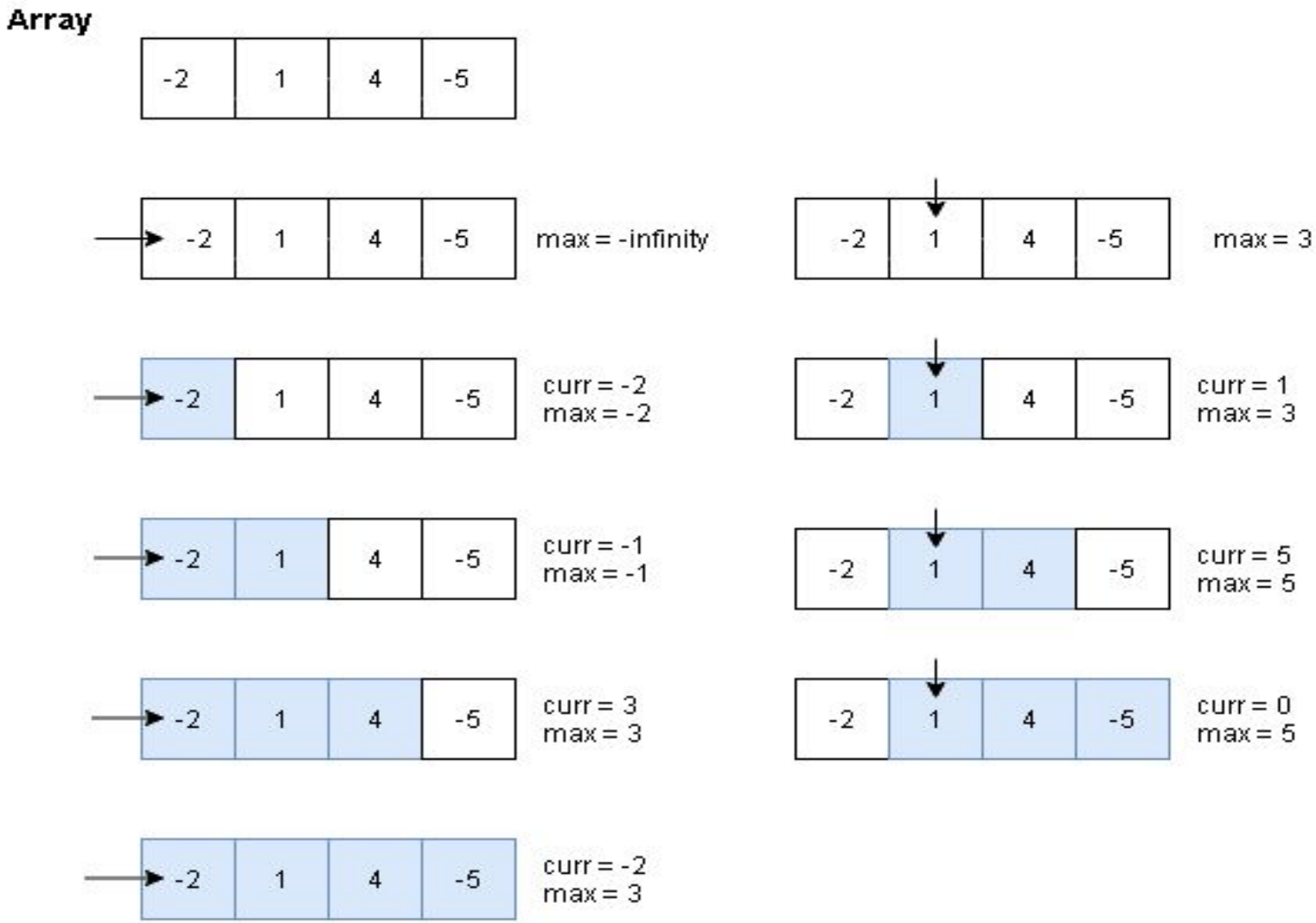
4 + (-1) + (-2) + 1 + 5 = 7

Maximum Contiguous Array Sum is 7

Maximum Subarray Sum



Maximum Subarray Sum



Complexity : $O(N^2)$

Maximum Subarray Sum

- Divide the given array in two halves
- Return the maximum of following three
  - Maximum subarray sum in left half (Make a recursive call)
  - Maximum subarray sum in right half (Make a recursive call)
  - Maximum subarray sum such that the subarray crosses the midpoint

- Time Complexity: $T(n) = 2T(n/2) + \Theta(n)$

Maximum Subarray Sum

- Maximum subarray sum such that the subarray crosses the midpoint

```
for (int i = mid; i >= low; i--)
{
        sum = sum + arr[i];
        if (sum > leftpartsum)
                leftpartsum = sum;
}
```

```
for (int i = mid+1; i <= high; i++)
{
        sum = sum + arr[i];
        if (sum > rightpartsum)
                rightpartsum = sum;
}
```

return lefpartsum + rightpartsum

Maximum Subarray Sum

- Time Complexity: $T(n) = 2T(n/2) + \Theta(n)$

Maximum Subarray Sum (Kadane's Algorithm)

- Local Max: Max possible value including the i^{th} element.
  - Max(A[i], A[i] + localmax[i-1])

Do we need an array to store localmax?

Maximum Subarray Sum (Kadane's Algorithm)

- Local Max: Max possible value including the i^{th} element.
  - Max(A[i], A[i] + localmax[i-1])

Do we need an array to store localmax?

- Yes, if we don't keep the track of global max
- No, for the previous approach

Maximum Subarray Sum (Kadane's Algorithm)

- Local Max: Max possible value including the i^{th} element.
  - Max(A[i], A[i] + localmax[i-1])
- Global Max: Update every time localmax is calculated

```
for (i=0; i < n; i++){
        localmax = max(A[i], A[i] + localmax);
        if(localmax > globalmax)
                globalmax = localmax;
}
```

Maximum Subarray Sum (Kadane's Algorithm)

```
for (i=0; i < n; i++){
        localmax = max(A[i], A[i] + localmax);
        if(localmax > globalmax)
    globalmax = localmax;
}
```

```
for (i=0; i < n; i++){
        localmax = A[i] + localmax;
        if(localmax > globalmax)
         globalmax = localmax;
        if(localmax < 0)
                localmax = 0;
}
```